Theory of Computation

Lecture 6: Non-Regular Languages
The Pumping Lemma

Sheikh Azizul Hakim
Lecturer, Department of CSE, BUET
Last compiled: July 18, 2026

Finite automata are powerful.

But finite memory has fundamental mathematical limits.

Today's Roadmap

1. **The Graph Intuition:** Why finite states force repetition.
2. **The Formal Theorem:** Sipser's Pumping Lemma.
3. **The Adversary Game:** How to navigate the nested quantifiers.
4. **The Proof Template:** A rigorous, step-by-step recipe for contradictions.
5. **Worked Examples:** Proving languages are non-regular.

A Suspicious Language

Consider the language of perfectly balanced binary strings:

$$B = \{0^n 1^n \mid n \geq 0\}$$

To accept 00001111, a machine must somehow "remember" that it saw exactly four 0s so it can verify the four 1s.

The problem: A DFA only has a fixed number of states (say, p states). What happens when we feed it $p + 1$ zeros?

The Pigeonhole Principle in Directed Graphs

A DFA is simply a directed graph. Reading a string is just tracing a walk through that graph.

Pigeonhole Principle: If you put $m + 1$ pigeons into m holes, at least one hole contains two pigeons.

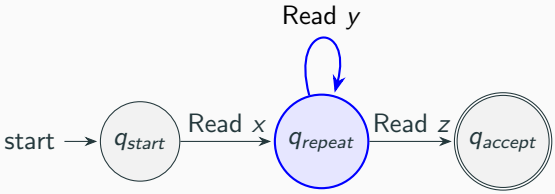
If a DFA has p states, and it reads a string of length p or greater, it visits at least $p + 1$ states (counting the start state). Therefore, **it must visit at least one state twice.**

Revisiting a State Creates a Cycle

In graph theory, a walk that intersects itself contains a cycle.

Let the string be $s = xyz$.

- x: The path from the start state to the first time we hit the repeated state.
- y: The cycle (leaving the state and returning to it).
- z: The path from the repeated state to the accept state.



The “Pump”

Because y is a cycle, the DFA doesn't actually know how many times it loops. If the DFA accepts xyz, it has no choice but to also accept:

- xz (looping 0 times, skipping y)
- $xyyz$ (looping 2 times)
- $xyyyz$ (looping 3 times)

Core Intuition: Any sufficiently long string accepted by a regular language MUST contain a cycle that can be “pumped” (repeated) infinitely without leaving the language.

The Pumping Lemma for Regular Languages

If A is a regular language, then there exists a number p (the pumping length) such that every string $s \in A$ with $|s| \geq p$ can be divided into three pieces, $s = xyz$, satisfying the following three conditions:

1. for each $i \geq 0$, $xy^i z \in A$,
2. $|y| > 0$, and
3. $|xy| \leq p$.

Why $|y| > 0$*?* The cycle must consume at least one character, or we aren't pumping anything.
Why $|xy| \leq p$*?* By the Pigeonhole Principle, the first duplicate state MUST occur within the first p steps.

The Logic of Contradiction

The Pumping Lemma is an implication:

$$\text{IF } A \text{ is regular} \implies A \text{ has the pumping property.}$$

By modus tollens, if A does **not** have the pumping property, it is **not** regular.

The lemma relies on heavily nested quantifiers:

$$\exists p, \forall s, \exists x, y, z, \forall i \ldots$$

To prove a language is NOT regular, we must negate this statement:

$$\forall p, \exists s, \forall x, y, z, \exists i \ldots$$

How to play The Adversary Game

Think of the proof as a turn-based game against the SETUP DEMON who claims A is regular. You win if you force a contradiction.

Turn	Action
1. Demon:	Chooses the pumping length p. (You don't know p, it's a variable).
2. YOU:	Choose a target string $s \in A$ such that $\|s\| \geq p$.
3. Demon:	Divides your string into $s = xyz$ however they want, as long as $\|y\| > 0$ and $\|xy\| \leq p$.
4. YOU:	Choose a pump count $i \geq 0$ such that $xy^i z \notin A$.

Rule of Rigor: You cannot choose the split (x, y, z). You must prove your i works for *any* valid split the Demon could possibly make.

The Rigorous 5-Step Proof Template

Every valid Pumping Lemma proof follows exactly this structure. **Do not skip steps.**

1. **Assume for contradiction** that language L is regular.
2. Let p be the pumping length given by the Pumping Lemma.
3. **Choose a specific string** $s \in L$ where $|s| \geq p$.
   (This is your creative step. Express s in terms of p.)
4. State that by the lemma, s can be split into $s = xyz$ such that $|y| > 0$ and $|xy| \leq p$.
   Analyze what y **must look like** based on the $|xy| \leq p$ constraint.
5. **Pump** the string. Choose an $i \geq 0$. Show mathematically that $xy^iz \notin L$, creating a contradiction. Conclude L is not regular.

Example 1: Matching Two Blocks

$$L_1 = \{0^n 1^n \mid n \geq 0\}.$$

1. **Assume for contradiction** that L_1 is regular.
2. Let p be its pumping length.
3. Choose
$$s = 0^p 1^p.$$
Clearly, $s \in L_1$ and $|s| = 2p \geq p$.
4. For every valid split $s = xyz$ with $|xy| \leq p$ and $|y| > 0$, both x and y lie inside the first block of 0s. Hence
$$y = 0^k \qquad \text{for some } 1 \leq k \leq p.$$

Example 1: Pumping Breaks Equality

5. Choose $i = 0$. Then

$$xy^0z = xz = 0^{p-k}1^p.$$

The pumped string contains $p - k$ zeros but still contains p ones.

Since $k > 0$,

$$p - k \neq p.$$

Therefore,

$$xz \notin L_1.$$

This contradicts the Pumping Lemma. Therefore, L_1 is not regular. ■

Intuition
The machine is forced to repeat or remove some early 0s without changing the later 1s.

Example 2: Palindromes

$$L_2 = \{w \in \{a, b\}^* \mid w = w^R\}.$$

1. **Assume for contradiction** that L_2 is regular.
2. Let p be its pumping length.
3. Choose
$$s = a^p b a^p.$$
This string is a palindrome and $|s| = 2p + 1 \geq p$.
4. For every valid split $s = xyz$, the condition $|xy| \leq p$ forces y to lie entirely inside the first block of as. Thus,
$$y = a^k \qquad \text{for some } 1 \leq k \leq p.$$

Example 2: Pumping Breaks Symmetry

5. Choose $i = 0$. Then

$$xy^0z = a^{p-k}ba^p.$$

The left side of the central b has $p - k$ copies of a, while the right side has p copies.

Because $k > 0$, the two sides are unequal. Hence the string is not a palindrome:

$$xy^0z \notin L_2.$$

This contradicts the Pumping Lemma. Therefore, L_2 is not regular. ■

Intuition
Pumping changes only the left half, so the mirror image no longer matches.

Example 3: Synchronizing Three Blocks

$$L_3 = \{a^n b^n c^n \mid n \geq 0\}.$$

1. **Assume for contradiction** that L_3 is regular.
2. Let p be its pumping length.
3. Choose
$$s = a^p b^p c^p.$$
Clearly, $s \in L_3$ and $|s| = 3p \geq p$.
4. For every valid split $s = xyz$, $|xy| \leq p$ places y completely inside the initial block of as. Therefore,
$$y = a^k \qquad \text{for some } 1 \leq k \leq p.$$

Example 3: One Block Changes, Two Do Not

5. Choose $i = 2$. Then

$$xy^2z = a^{p+k}b^pc^p.$$

The numbers of as, bs, and cs are now

$$p + k, \quad p, \quad p.$$

Since $k > 0$, these three quantities are not equal. Thus,

$$xy^2z \notin L_3.$$

This contradicts the Pumping Lemma. Therefore, L_3 is not regular. ■

Intuition
A finite automaton cannot keep three arbitrarily large blocks perfectly synchronized.

Example 4: Copying Across a Separator

$$\Sigma = \{0, 1, \#\} \qquad L_4 = \{w\#w \mid w \in \{0,1\}^*\}.$$

1. **Assume for contradiction** that L_4 is regular.
2. Let p be its pumping length.
3. Choose
$$s = 0^p 1 \# 0^p 1.$$
Here the same word $0^p 1$ appears on both sides of $\#$, so $s \in L_4$. Also, $|s| = 2p + 3 \geq p$.
4. For every valid split $s = xyz$, $|xy| \leq p$ forces y into the first block of 0s. Hence
$$y = 0^k \qquad \text{for some } 1 \leq k \leq p.$$

Example 4: Pumping Breaks the Copy

5. Choose $i = 0$. Then

$$xy^0z = 0^{p-k}1\#0^p1.$$

The word to the left of $\#$ is now $0^{p-k}1$, while the word to the right is still 0^p1.

Since $k > 0$, the two words are different. Therefore,

$$xy^0z \notin L_4.$$

This contradicts the Pumping Lemma. Therefore, L_4 is not regular. ■

Intuition
Pumping edits the first copy but leaves the second copy untouched.

Example 5: Unary Strings of Square Length

$$L_5 = \{a^{n^2} \mid n \geq 0\}.$$

1. **Assume for contradiction** that L_5 is regular.
2. Let $p \geq 1$ be its pumping length.
3. Choose
$$s = a^{p^2}.$$
Since p^2 is a perfect square, $s \in L_5$, and $|s| = p^2 \geq p$.
4. Every valid split $s = xyz$ has
$$y = a^k \qquad \text{for some } 1 \leq k \leq p.$$

Example 5: Pumping Lands Between Two Squares

5. Choose $i = 2$. The pumped string has length

$$|xy^2z| = p^2 + k.$$

Since $1 \leq k \leq p$,

$$p^2 < p^2 + k \leq p^2 + p < (p+1)^2.$$

Thus, $p^2 + k$ lies strictly between the consecutive squares p^2 and $(p+1)^2$.

Therefore, $p^2 + k$ is not a perfect square, so

$$xy^2z \notin L_5.$$

This contradicts the Pumping Lemma. Therefore, L_5 is not regular. ■

Intuition
Even over a one-symbol alphabet, a DFA cannot recognize widely growing gaps between valid lengths.

What the Five Examples Teach Us

Language type	What must be remembered	What pumping breaks
0^n1^n	one count	equality of two blocks
Palindromes	distant symbols	left–right symmetry
$a^n b^n c^n$	three counts	three-way synchronization
$w\#w$	an entire word	exact copying
a^{n^2}	special lengths	arithmetic spacing

Although the languages look different, every proof uses the same five moves:

$$\text{assume} \;\to\; p \;\to\; s \;\to\; \text{trap } y \;\to\; \text{pump}.$$

Checklist for Student Proofs

When grading your own proofs, ask:

1. Did I say "let p be the pumping length"? (Do not invent a value such as $p = 5$.)
2. Did I choose a string s using p?
3. Did I verify both $s \in L$ and $|s| \geq p$?
4. Did I use $|xy| \leq p$ to prove where y must occur?
5. Did I handle **every** valid split rather than choosing y myself?
6. Did I select an i and clearly prove that $xy^i z \notin L$?

Closure Proof 1: When Pumping Becomes Tedious

Consider

$$L_{\neq} = \{0^m 1^n \mid m, n \geq 0,\ m \neq n\}.$$

A direct pumping argument is awkward because pumping may preserve the inequality. Closure properties reveal the problem immediately.

- Assume that $L_{\neq}$ is regular.
- The language
$$R = 0^* 1^*$$
is regular.
- Regular languages are closed under difference.
- Therefore, $R \setminus L_{\neq}$ must be regular.
- However,
$$R \setminus L_{\neq} = \{0^n 1^n \mid n \geq 0\},$$
which is not regular.

Contradiction. Therefore, $L_{\neq}$ is not regular.

Closure Proof 2: When Pumping Becomes Tedious

Consider

$$L_< = \{0^m 1^n \mid m < n\}.$$

Assume that $L_<$ is regular.

- Regular languages are closed under reversal.
- Swap 0 and 1 after reversing the strings.
- We would then obtain the regular language

$$L_> = \{0^m 1^n \mid m > n\}.$$

- Hence, the union

$$L_< \cup L_> = \{0^m 1^n \mid m \neq n\}$$

  would be regular.
- Since 0^*1^* is regular, its difference with this union would also be regular:

$$0^*1^* \setminus (L_< \cup L_>) = \{0^n 1^n \mid n \geq 0\}.$$

This is impossible. Therefore, $L_<$ is not regular.

More Pumping-Lemma Examples I

Language	Choose s	Forced form of y	Pump and contradiction
$\{a^{n+1}b^n \mid n \geq 0\}$	$a^{p+1}b^p$	$y = a^k,\ k > 0$	Choose $i = 0$. Then $a^{p+1-k}b^p$ does not have exactly one more a than b.
$\{a^m b^n \mid m > n\}$	$a^{p+1}b^p$	$y = a^k,\ k > 0$	Choose $i = 0$. Then $p+1-k \leq p$, so the number of as is no longer greater than the number of bs.
$\{a^n b^{2n} \mid n \geq 0\}$	$a^p b^{2p}$	$y = a^k,\ k > 0$	Choose $i = 0$. The counts become $p-k$ and $2p$, but $2p \neq 2(p-k)$.
$\{a^m b^n c^{m+n} \mid m, n \geq 0\}$	$a^p b^p c^{2p}$	$y = a^k,\ k > 0$	Choose $i = 0$. The first two blocks total $2p-k$, while the final block still has length $2p$.
$\{a^i b^j c^k \mid i = j \text{ or } j = k\}$	$a^p b^p c^{p+1}$	$y = a^r,\ r > 0$	Choose $i = 0$. Now $p - r \neq p$ and $p \neq p+1$, so neither equality holds.

More Pumping-Lemma Examples II

Language	Choose s	Forced form of y	Pump and contradiction
$\{u\#v \mid \lvert u\rvert = \lvert v\rvert\}$	$0^p\#1^p$	$y = 0^k$, $k > 0$	Choose $i = 0$. The two sides have lengths $p - k$ and p, so they are unequal.
$\{a^q \mid q \text{ is prime}\}$	a^q, where $q > p$ is prime	$y = a^k$, $1 \le k \le p$	Choose $i = q + 1$. The new length is $q + qk = q(k + 1)$, which is composite.
$\{a^{2^n} \mid n \ge 0\}$	a^{2^m}, where $2^m > p$	$y = a^k$, $1 \le k \le p$	Choose $i = 2$. Then $2^m < 2^m + k < 2^{m+1}$, so the new length is not a power of two.
$\{a^m b^n \mid m = n \text{ or } m = 2n\}$	$a^p b^p$	$y = a^k$, $k > 0$	Choose $i = 0$. Since $p - k < p$, the new number of as is neither p nor $2p$.
$\{a^m b^{m+n} c^n \mid m, n \ge 0\}$	$a^p b^{2p} c^p$	$y = a^k$, $k > 0$	Choose $i = 0$. The two outer blocks total $2p - k$, but the middle block still has length $2p$.

Problems to Try at Home

For each language below, use the **five-step Pumping Lemma template** to prove that it is not regular.

- $L_1 = \{0^m 1^n \mid m \neq n\}$.
- $L_2 = \{0^m 1^n \mid \gcd(m, n) = 1\}$.
- $L_3 = \{0^m 1^n \mid m \mid n,\ m, n \geq 1\}$.
- $L_4 = \{(ab)^n c^n \mid n \geq 0\}$.
- $L_5 = \{a^{n!} \mid n \geq 1\}$.
- $L_6 = \{a^{F_n} \mid n \geq 0\}$, where F_n is the nth Fibonacci number.
- Let L_7 be the language of all correctly balanced parenthesis strings over $\{(,)\}$.

Example 1: Unequal Numbers of 0's and 1's

Consider

$$L_{\neq} = \{w \in \{0,1\}^* \mid \#_0(w) \neq \#_1(w)\},$$

where $\#_\sigma(w)$ denotes the number of occurrences of the symbol σ in w.

We prove that $L_{\neq}$ is not regular using closure properties.

1. Assume for contradiction that $L_{\neq}$ is regular.
2. Regular languages are closed under complement. Therefore,

   $$E = \{0,1\}^* \setminus L_{\neq} = \{w \in \{0,1\}^* \mid \#_0(w) = \#_1(w)\}$$

   must be regular.
3. The language

   $$R = 0^*1^*$$

   is regular.

Example 1: Restricting the Symbol Order

Regular languages are closed under intersection. Therefore, $E \cap R$ must be regular.

However,

$$\begin{aligned} E \cap R &= \{w \in 0^*1^* \mid \#_0(w) = \#_1(w)\} \\ &= \{0^n1^n \mid n \geq 0\}. \end{aligned}$$

But

$$\{0^n1^n \mid n \geq 0\}$$

is not regular.

This is a contradiction. Therefore,

$$\boxed{L_{\neq} \text{ is not regular.}}$$

■

Main idea
The intersection with 0^*1^* forces all 0's to occur before all 1's, exposing the familiar language 0^n1^n.

Example 1: Common Errors and Misconceptions

Potential mistakes

- The original language contains strings in any order:

  $$0101, \quad 1100, \quad 1010, \quad \ldots$$

  It is not restricted to strings of the form 0^m1^n.

- The complement of $L_{\neq}$ is

  $$\{w \mid \#_0(w) = \#_1(w)\},$$

  not immediately $\{0^n1^n \mid n \geq 0\}$.

- We obtain $\{0^n1^n \mid n \geq 0\}$ only after intersecting with 0^*1^*.

- A complement must always be taken relative to a fixed universe. Here the universe is $\{0,1\}^*$.

Example 2: Unary Strings of Prime Length

Consider

$$L_{\text{prime}} = \{a^t \mid t \text{ is prime}\}.$$

1. Assume for contradiction that L_{prime} is regular.
2. Let p be the pumping length.
3. Since there are infinitely many primes, choose a prime $q > p$, and let

   $$s = a^q.$$

   Then $s \in L_{\text{prime}}$ and $|s| = q \geq p$.
4. For every valid decomposition $s = xyz$ satisfying

   $$|xy| \leq p \qquad \text{and} \qquad |y| > 0,$$

   we have

   $$y = a^k \qquad \text{for some } 1 \leq k \leq p.$$

Example 2: Pumping Produces a Composite Length

Choose

$$i = q + 1.$$

Since the original string contains one copy of y, replacing it by $q + 1$ copies adds qk symbols. Therefore,

$$\begin{aligned} |xy^{q+1}z| &= q + qk \\ &= q(k + 1). \end{aligned}$$

Now,

$$q \geq 2 \qquad \text{and} \qquad k + 1 \geq 2.$$

Thus $q(k + 1)$ is a product of two integers greater than 1, so it is composite. Hence,

$$xy^{q+1}z \notin L_{\text{prime}}.$$

This contradicts the Pumping Lemma. Therefore,

$$\boxed{L_{\text{prime}} \text{ is not regular.}}$$

Example 2: Common Errors and Misconceptions

Potential mistakes

- The pumping length p is an arbitrary positive integer. It is not necessarily prime.
- We do not choose $s = a^p$, because p may be composite. Instead, after receiving p, we choose a separate prime

$$q > p.$$

- We do not choose y. The adversary chooses the split. We only conclude that

$$y = a^k, \qquad 1 \leq k \leq p.$$

- Pumping with $i = q + 1$ does not make the length $(q + 1)k$. The correct length is

$$q + (i - 1)k = q + qk = q(k + 1).$$

- The pumping exponent i may depend on the pumping length and the chosen string. It does not have to be a fixed constant.

Example 3: Unary Strings of Composite Length

Consider

$$L_{\text{comp}} = \{a^t \mid t \text{ is composite}\}.$$

Recall that 0 and 1 are neither prime nor composite. Therefore,

$$a^* = L_{\text{prime}} \cup L_{\text{comp}} \cup \{\varepsilon, a\}.$$

Assume for contradiction that L_{comp} is regular.

- The finite language $\{\varepsilon, a\}$ is regular.
- Hence,

  $$L_{\text{comp}} \cup \{\varepsilon, a\}$$

  would be regular.
- Regular languages are closed under difference.

Therefore,

$$a^* \setminus \big(L_{\text{comp}} \cup \{\varepsilon, a\}\big)$$

would be regular.

Example 3: Using the Earlier Prime Result

But

$$a^* \setminus \big(L_{\text{comp}} \cup \{\varepsilon, a\}\big) = L_{\text{prime}}.$$

Thus L_{prime} would be regular.

However, the previous example proved that

$$L_{\text{prime}} = \{a^t \mid t \text{ is prime}\}$$

is not regular.

This is a contradiction. Therefore,

$$\boxed{L_{\text{comp}} \text{ is not regular.}}$$

■

Main idea
Prime and composite lengths cover all positive unary lengths except length 1. We must also account for length 0, represented by ε.

Example 3: Common Errors and Misconceptions

Potential mistakes

- The complement of the composite-length language is not exactly the prime-length language.
- The strings
$$\varepsilon = a^0 \qquad \text{and} \qquad a = a^1$$
have lengths 0 and 1, which are neither prime nor composite.
- Therefore,
$$a^* \setminus L_{\text{comp}} = L_{\text{prime}} \cup \{\varepsilon, a\}.$$
- To isolate the prime-length language, we must remove both L_{comp} and $\{\varepsilon, a\}$:
$$L_{\text{prime}} = a^* \setminus \big(L_{\text{comp}} \cup \{\varepsilon, a\}\big).$$
- The proof relies on the earlier result that L_{prime} is not regular.

Example 4: Unary Strings of Fibonacci Length

Let

$$F_0 = 0, \qquad F_1 = 1, \qquad F_{n+1} = F_n + F_{n-1} \quad (n \geq 1),$$

and consider

$$L_{\mathrm{Fib}} = \{a^{F_n} \mid n \geq 0\}.$$

1. Assume for contradiction that L_{Fib} is regular.
2. Let p be its pumping length.
3. Since the Fibonacci numbers grow without bound, choose $m \geq 3$ sufficiently large that

   $$F_{m-1} > p.$$

   Let

   $$s = a^{F_m}.$$

   Since $F_m > F_{m-1} > p$, we have

   $$s \in L_{\mathrm{Fib}} \qquad \text{and} \qquad |s| \geq p.$$

Example 4: Pumping Between Consecutive Fibonacci Numbers

For every valid decomposition $s = xyz$,

$$y = a^k \qquad \text{for some } 1 \leq k \leq p.$$

Choose $i = 2$. Then

$$|xy^2z| = F_m + k.$$

Because $k \geq 1$, $\quad F_m < F_m + k$.

Also, since $k \leq p < F_{m-1}$, $\quad F_m + k < F_m + F_{m-1} = F_{m+1}$.

Therefore, $\quad F_m < F_m + k < F_{m+1}$.

Thus $F_m + k$ lies strictly between two consecutive Fibonacci numbers. It cannot itself be a Fibonacci number. Hence, $xy^2z \notin L_{\text{Fib}}$.

Therefore, $\boxed{L_{\text{Fib}} \text{ is not regular.}}$ ■

Example 4: Common Errors and Misconceptions

Potential mistakes

- The pumping length p need not be a Fibonacci number.
- We must choose a Fibonacci number after receiving p. Specifically, choose m such that

$$F_{m-1} > p.$$

- Merely choosing $F_m > p$ is not enough for this proof. We need

$$k \leq p < F_{m-1}$$

so that the pumped length remains below F_{m+1}.

- The first few Fibonacci numbers contain a repetition: $F_1 = F_2 = 1$. Choosing $m \geq 3$ avoids this irrelevant initial duplication.
- We pump upward with $i = 2$. Pumping down may accidentally produce another Fibonacci length and requires more analysis.

Example 5: Equal Numbers of *ab* and *ba*

Let

$$\Sigma = \{a, b, c\},$$

and define

$$L_{ab=ba} = \{w \in \Sigma^* \mid \#_{ab}(w) = \#_{ba}(w)\},$$

where $\#_{ab}(w)$ denotes the number of occurrences of ab as a consecutive substring of w.

Occurrences may overlap. For example,

$$w = aba$$

contains one occurrence of ab and one occurrence of ba, so

$$aba \in L_{ab=ba}.$$

Similarly,

$$abcba$$

contains one ab and one ba, while

$$abcab$$

contains two occurrences of ab and no occurrence of ba.

Example 5: Restricting to a Two-Block Language

For

$$w = (abc)^m(bac)^n,$$

we have

$$\#_{ab}(w) = m \qquad \text{and} \qquad \#_{ba}(w) = n.$$

Therefore,

$$L_{ab=ba} \cap R = \{(abc)^n(bac)^n \mid n \geq 0\}.$$

Assume for contradiction that $L_{ab=ba}$ is regular.

Since R is regular and regular languages are closed under intersection, the language

$$K = \{(abc)^n(bac)^n \mid n \geq 0\}$$

would also be regular.

We now prove directly, using the Pumping Lemma, that K is not regular.

Example 5: Setting Up the Pumping Argument

Assume that

$$K = \{(abc)^n(bac)^n \mid n \geq 0\}$$

is regular, and let p be its pumping length.

Choose

$$s = (abc)^p(bac)^p.$$

Clearly,

$$s \in K \qquad \text{and} \qquad |s| = 6p \geq p.$$

For every valid decomposition $s = xyz$ satisfying $|xy| \leq p \qquad$ and $\qquad |y| > 0$, the substring y lies entirely within the first p symbols of $(abc)^p$.

Since the complete first region $(abc)^p$ has length $3p$, the pumped part cannot reach the second region $(bac)^p$.

Let $|y| = k, \qquad 1 \leq k \leq p$.

We choose $i = 0$ and consider $xy^0z = xz$.

Example 5: Case 1 — $3 \nmid k$

Suppose first that

$$3 \nmid k.$$

The original string has length

$$|s| = 6p.$$

After pumping down,

$$|xz| = 6p - k.$$

Because $6p$ is divisible by 3, but k is not divisible by 3, $3 \nmid (6p - k)$.

However, every string in K has the form $(abc)^n(bac)^n$ and therefore has length $3n + 3n = 6n$, which is divisible by 3.

Consequently, $xz \notin K$.

Thus, in this case, pumping down produces a string outside the language.

Example 5: Case 2 — $3 \mid k$

Now suppose that

$$3 \mid k.$$

Then

$$k = 3r$$

for some integer $r \geq 1$.

The word $(abc)^p$ is periodic with period 3. Removing any consecutive $3r$ symbols preserves the abc-phase of the remaining symbols.

Therefore, after pumping down,

$$xz = (abc)^{p-r}(bac)^p.$$

The first region now contains $p - r$ copies of abc, while the second region still contains p copies of bac.

Since $r \geq 1$, $p - r \neq p$.

Hence xz is not of the required form $(abc)^n(bac)^n$.

Therefore, $xz \notin K$.

Example 5: Completing the Proof

For every valid decomposition $s = xyz$, let $k = |y| > 0$.

There are only two possibilities:

$$3 \nmid k \implies |xz| \text{ is not divisible by 3,}$$
$$3 \mid k \implies xz = (abc)^{p-r}(bac)^p \text{ for some } r \geq 1.$$

In both cases, $xz \notin K$.

This contradicts the Pumping Lemma. Therefore, $K = \{(abc)^n(bac)^n \mid n \geq 0\}$ is not regular.

But if $L_{ab=ba}$ were regular, then $K = L_{ab=ba} \cap R$ would be regular. This is impossible.

Therefore, $\boxed{L_{ab=ba} \text{ is not regular over } \{a, b, c\}.}$ ■

Example 5: Common Errors and Misconceptions

Potential mistakes

- The pumping length p counts individual symbols, not complete abc-blocks.
- From $|xy| \leq p$, we may conclude that y lies inside the first $(abc)^p$ region. We may not conclude that y is itself a sequence of complete abc-blocks.
- The adversary chooses x, y, and z. Therefore, we cannot assume that y begins at the start of an abc-block.
- We must consider both possibilities:

$$3 \nmid |y| \qquad \text{and} \qquad 3 \mid |y|.$$

- If $3 \nmid |y|$, the simplest contradiction comes from the total length. There is no need to analyze every resulting symbol boundary.
- We may write $xz = (abc)^{p-r}(bac)^p$ only in the case where $|y| = 3r$.
- Proving that K is non-regular is not by itself the final conclusion. We must use $K = L_{ab=ba} \cap R$ and closure under intersection to conclude that $L_{ab=ba}$ is non-regular.

Why the Extra Symbol c Matters

Over the binary alphabet $\{a, b\}$, the difference

$$\#_{ab}(w) - \#_{ba}(w)$$

depends only on the first and last symbols of w:

$$\#_{ab}(w) - \#_{ba}(w) = \begin{cases} 1, & \text{if } w \text{ starts with } a \text{ and ends with } b, \\ -1, & \text{if } w \text{ starts with } b \text{ and ends with } a, \\ 0, & \text{otherwise.} \end{cases}$$

Thus equality can be checked using finite memory over $\{a, b\}$.

Over $\{a, b, c\}$, occurrences of c separate the word into many binary segments. Each segment may contribute -1, 0, or 1 to the difference, and these contributions may accumulate without bound.

Conclusion

The language is regular over $\{a, b\}$, but it is not regular over $\{a, b, c\}$.

Lessons from the Worked-Out Examples

Language	Main proof technique	Key danger
Unequal numbers of 0's and 1's	Complement and intersection	Confusing all strings with 0^*1^*
Prime unary length	Pumping Lemma	Assuming the pumping length is prime
Composite unary length	Closure under difference	Forgetting lengths 0 and 1
Fibonacci unary length	Pumping between consecutive values	Assuming p is Fibonacci
Equal numbers of ab and ba	Intersection	Ignoring block boundaries

The proof method must match the structure of the language.

Summary

- **Intuition:** Finite states and sufficiently long inputs force cycles.
- **Theorem:** Every sufficiently long string in a regular language must be pumpable.
- **Proof strategy:** Choose a string that traps the pumpable part in a vulnerable region.
- **Rigor:** The argument must work for every valid decomposition $s = xyz$.
- **Alternative:** When direct pumping becomes awkward, closure properties may be cleaner.

Next Lecture: Context-Free Grammars and the idea of recursive structure.